EEB3

BIG CLOCK

---

220 V Bulb 7-Segment Display
Driven by 4 x 8-Relay Modules
Arduino MEGA 2560 + DS3231 RTC

Prepared by: ***Taqi Abbas***
Location: European School of Brussels, Ixelles
Required library: RTClib by Adafruit (v2.x)

1. SYSTEM OVERVIEW

What this clock does

A large wooden-frame wall clock for European School of Brussels, Ixelles. Each segment of each digit is an individual 220 V light bulb. Four 8-relay modules switch the bulbs (one module per digit). A DS3231 real-time clock module provides accurate timekeeping. An Arduino MEGA 2560 is the controller.

Display layout — HH:MM

	Module	Mega pins	Notes
	`Module 1`	`22–29`	IN8 (pin 29) = colon dots
	`Module 2`	`30–37`	IN8 (pin 37) unused
	`Module 3`	`38–45`	IN8 (pin 45) unused
	`Module 4`	`46–53`	IN8 (pin 53) unused

Relay-to-segment mapping (same for every module)

Each relay IN pin drives one bulb segment. The frame label e.g. 2.3 means digit 2, relay 3.

	Position on digit	Frame label
	Top-left vertical	`x.1`
	Top horizontal	`x.2`
	Top-right vertical	`x.3`
	Middle horizontal	`x.4`
	Bottom-left vertical	`x.5`
	Bottom-right vertical	`x.6`
	Bottom horizontal	`x.7`
	Colon dots (Module 1 only)	`1.8`

2. HARDWARE LIST

	Qty
	1
	1
	4
	1
	1
	30
	as needed
	1 spare
	~50
	1

3. CONNECTIONS

3a. Power architecture

Relay coils draw up to 2.5 A combined — far beyond what the Mega USB or onboard 5 V regulator can supply. Use a separate external 5 V / 3 A supply for the relay boards. The Mega 5 V pin powers only the RTC (~2 mA).

	From	To	Note
	External 5V (+)	All 4 boards VCC and JD-VCC	Daisy-chain or power rail
	External 5V (–)	All 4 boards GND	Must also join Mega GND
	Mega GND near pin 53	Common ground rail	Ties both supplies together
	Mega 5V pin	RTC VCC	Only ~2 mA — safe from Mega
	Mega GND	RTC GND	
	USB or 7-12 V barrel	Mega board only	Independent of relay supply

3b. RTC (DS3231) — 4 wires

	Notes
	Low current — Mega regulator is fine here
	Hardware I2C — only valid SDA pin on MEGA
	Hardware I2C — only valid SCL pin on MEGA

Pins 20 and 21 are the ONLY hardware I2C pins on the Mega. Do not use any other pins.

3c. Signal wires — 32 wires total

All signal wires connect to the large double-row header at the top of the Mega (pins 22–53). Pin numbers are printed on the board.

Module 1 — HOURS TENS + colon on IN8

gment	Frame label
top-left vertical	1.1
top horizontal	1.2
top-right vertical	1.3
middle horizontal	1.4
bottom-left vertical	1.5
bottom-right vertical	1.6
bottom horizontal	1.7
n dots	1.8

Module 2 — HOURS UNITS (IN8 unused)

gment	Frame label
top-left vertical	2.1
top horizontal	2.2

- top-right vertical	2.3
- middle horizontal	2.4
- bottom-left vertical	2.5
- bottom-right vertical	2.6
- bottom horizontal	2.7
sed — leave empty	–

Module 3 — MINUTES TENS (IN8 unused)

gment	Frame label
top-left vertical	3.1
- top horizontal	3.2
- top-right vertical	3.3
- middle horizontal	3.4
- bottom-left vertical	3.5
- bottom-right vertical	3.6
- bottom horizontal	3.7
sed — leave empty	–

Module 4 — MINUTES UNITS (IN8 unused)

gment	Frame label
top-left vertical	4.1
- top horizontal	4.2
- top-right vertical	4.3
- middle horizontal	4.4
- bottom-left vertical	4.5
- bottom-right vertical	4.6
- bottom horizontal	4.7
sed — leave empty	–

3d. 220 V mains side

Wire each bulb through COM to NO so it is off at power-up.

ct to	Relay OFF	Relay ON
LIVE wire	—	—
rminal A	Open — bulb off	Closed — bulb on
unconnected	Closed	Open

WARNING: 220 V wiring must be installed and verified by a qualified electrician. All mains wiring must be fused and enclosed. Keep mains and low-voltage wiring physically separated.

4. SETUP AND UPLOAD WORKFLOW

Arduino IDE settings (do once, never change)

	Value
	Arduino Mega or Mega 2560
	ATmega2560 (Mega 2560)
	/dev/cu.usbmodem101 or /dev/cu.usbserial-XXXX
	COM3, COM4 … check Device Manager
	RTClib by Adafruit — install via Library Manager

Every time you upload — exact two-step workflow

The clock sets its own time automatically. No manual UTC entry, no typing, no extra tools to install. Your computer clock (NTP-synced) is the source of truth.

	What happens
normal)	IDE compiles the sketch and uploads it to the Mega. Takes about 10–15 second
mand in Finder	A Terminal window opens. Script connects to the Arduino, reads exact UTC from second. Window shows confirmation then closes.
bat in File Explorer	A Command Prompt opens. Same process as Mac. RTC is set to within 1 secon

Important: Steps 1 and 2 overlap in time. You do NOT need to wait for Step 1 to finish before starting Step 2. Double-click the file while the upload bar is still moving — the script waits up to 30 seconds for the board to boot.

What if you skip Step 2?

If the script is not run within 5 seconds of the board booting, the Arduino automatically falls back to the compile-time stamp from your computer clock, converted to UTC. The clock will run but may be 10–20 seconds behind. For day-to-day use this is perfectly acceptable. Run Step 2 any time to correct it precisely.

Project files — keep all in the same folder

	Purpose	Platform
o	Arduino sketch — open and upload with IDE	Both
	Double-click after upload to set exact time	Mac only
	Double-click after upload to set exact time	Windows only
	The actual time-setter — do not move or rename	Both
f	This document	Both

First time on a new computer: the .command / .bat file installs the required pyserial library automatically. No manual pip install needed.

Coin cell replacement

The DS3231 uses a CR2032 cell. Typical life is 5–8 years. After replacement simply upload the sketch and run the time-setter script. The clock will be accurate within seconds.

5. DISPLAY BEHAVIOUR

Operating hours — relay rest cycle

All 32 relays are completely switched OFF outside school hours. This gives the relay contacts a long daily rest, dramatically extending the lifespan of both the relays and the bulbs. The transition happens exactly once per boundary — no repeated clicking.

Display	All relays
Nothing — completely blank	OFF
Active (see table below)	Normal operation
Nothing — completely blank	OFF

Display logic during operating hours (08:00–17:00)

EEb3 and the period label appear ONLY during an active period (P1–P9). During the break, between periods, before P1, and after P9 the clock shows plain time the entire minute with no flash.

	Second :00–:57	Second :58	Second :59
P9)	HH:MM colon ON	EEb3 colon OFF	P x colon OFF
	HH:MM colon ON	HH:MM colon ON	HH:MM colon ON
aps	HH:MM colon ON	HH:MM colon ON	HH:MM colon ON
:30)	HH:MM colon ON	HH:MM colon ON	HH:MM colon ON
0)	HH:MM colon ON	HH:MM colon ON	HH:MM colon ON

Bell schedule — local Brussels time

	End	What the clock shows
	08:30	HH:MM only — no flash
	09:15	HH:MM, then EEb3 at :58, P 1 at :59
	09:20	HH:MM only — no flash
	10:05	HH:MM, then EEb3 at :58, P 2 at :59
	10:10	HH:MM only — no flash
	10:55	HH:MM, then EEb3 at :58, P 3 at :59
	11:15	HH:MM only — no flash
	12:00	HH:MM, then EEb3 at :58, P 4 at :59
	12:05	HH:MM only — no flash
	12:50	HH:MM, then EEb3 at :58, P 5 at :59
	13:00	HH:MM only — no flash
	13:45	HH:MM, then EEb3 at :58, P 6 at :59
	13:50	HH:MM only — no flash
	14:35	HH:MM, then EEb3 at :58, P 7 at :59
	14:40	HH:MM only — no flash
	15:25	HH:MM, then EEb3 at :58, P 8 at :59

	15:30	HH:MM only — no flash
	16:15	HH:MM, then EEb3 at :58, P 9 at :59
	17:00	HH:MM only — no flash
	08:00	All relays OFF — completely dark

To edit times: update schedule[] in the code. CLOCK_ON_MINS and CLOCK_OFF_MINS control the on/off hours. All times are LOCAL Brussels time in minutes from midnight.

6. DAYLIGHT-SAVING TIME LOGIC (BRUSSELS)

The RTC stores UTC permanently. Every second the code applies the EU DST rule mathematically — no lookup table, no manual intervention, valid for any future year.

	Starts	Ends
n	Last Sunday of March, 01:00 UTC	Last Sunday of October, 01:00 UTC
n	Last Sunday of October, 01:00 UTC	Last Sunday of March, 01:00 UTC

How the calculation works

lastSundayDay(year, month) takes the 31st of the month, reads its day-of-week (0 = Sunday), and subtracts that from 31. Both March and October have 31 days so no special case is needed. This gives the correct last-Sunday date for any year.

If the EU ever abolishes DST, change only one line: replace isEuSummerTime(utc) ? 2 : 1 with a fixed offset of 1 or 2.

7. SERIAL MONITOR DIAGNOSTICS

Open Serial Monitor at 9600 baud. Every time the display changes, one line is printed:

```
UTC 2026-06-09T10:05:32 Local 12:05 showing P 5 colon=off
```

	Meaning and action
	RTC not responding. Check VCC, GND, SDA, SCL wiring. Re-seat the module or try a re
	CR2032 coin cell flat or missing. Replace cell and re-set UTC using Section 4 Steps 1 to
	Check port selection and baud rate (9600). If watchdog keeps rebooting, the RTC is abs
	I2C glitch. Code auto-discards reads outside 2024-2099. If persistent, re-seat wiring and
	Watchdog reboots Mega within 8 s automatically. If it keeps recurring, check I2C wiring a

8. COMMON ISSUES AND CHECKS

	Most likely cause	Check / Fix
s	Short circuit in wiring	Unplug relay board power wires first. Test bare Mega plus USB. Add short reappears.
	CH340 USB driver missing	Download CH340 driver. Install the pkg file. Approve in System Sett Reboot Mac.
	Driver missing or wrong port	Open Device Manager. Look for yellow ! on a COM entry. Install CH
	Wire swapped or relay polarity wrong	Verify pin number against Section 3c. Toggle RELAY_ACTIVE_LOW
	SEG2RELAY or PINS mismatch	Check Serial Monitor for the frame being sent. Test each relay to co
	RTC set to local time instead of UTC	Re-set RTC using the correct UTC value. See Section 4.
	DS3231 normal tolerance (+/-2 ppm)	Under 1 minute per year is normal. Re-enter UTC if drift becomes no
	Loose relay terminal or failing relay	Inspect Module 1 IN8 terminal and Mega pin 29 connection.
	External 5V supply not powered on	Check relay supply. Measure voltage on relay VCC pin.
	Watchdog triggered by stalled loop	Almost always the RTC is not responding. Check I2C wiring.

9. RELIABILITY AND LONG-LIFE DESIGN NOTES

Daily relay rest — operating hours 08:00–17:00

All 32 relays are completely de-energised outside school hours. allRelaysOff() uses writeRelay() internally, so once the board is dark there are zero repeated clicks — only one transition at 08:00 and one at 17:00. That is ~15 hours of rest every day, dramatically extending relay and bulb lifespan. CLOCK_ON_MINS and CLOCK_OFF_MINS control the window.

Relay switching minimised

writeRelay() checks current state before driving the pin. A relay only clicks when its segment genuinely changes. Showing the same digit twice produces zero relay operations.

Colon stays on — never blinks

Blinking at 1 Hz would produce ~31 million operations per year on the colon relay alone. The colon stays steadily on during time display.

EEb3 and period label: active periods only

The :58 EEb3 flash and :59 period label are shown ONLY when a lesson is in progress. During the break, between periods, before P1, and after P9 the clock shows plain HH:MM the whole minute — no unnecessary relay toggles.

Hardware watchdog

wdt_enable(WDTO_8S) makes the microcontroller reboot itself if the main loop stops for more than 8 seconds, recovering from I2C hangs and unexpected lock-ups with no human intervention needed.

UTC-based timekeeping

The RTC stores UTC and is never adjusted for DST. The clock corrects itself on every DST boundary automatically, requiring no physical access and no buttons.

Defensive RTC reads

Any timestamp with a year outside 2024–2099 is silently discarded. This prevents corrupted I2C data from causing display errors.

Safe boot sequence

All relay pins are driven to the OFF level before being configured as outputs, eliminating the brief boot glitch that would otherwise flash all bulbs on every power cycle.

Coin cell maintenance

Replace the DS3231 CR2032 proactively every 5–7 years. After replacement, re-enter the UTC time once using Section 4.

10. FULL CODE LISTING

File: EEB3_Clock_Mega_Relay.ino — Board: Arduino Mega or Mega 2560, Processor: ATmega2560

```
/* ============================================================================
EEB3 BIG CLOCK - 220V bulb 7-segment digits driven by 4x 8-relay modules
----------------------------------------------------------------------------
Board : Arduino MEGA 2560
Clock : DS3231 RTC (I2C) - RTC is kept in UTC, local time is computed in
software so Brussels daylight-saving is handled WITHOUT ever touching
the hardware (no buttons needed). Good for 20+ years.
Display layout (HH:MM):
[Module 1] [Module 2] : [Module 3] [Module 4]
hours-tens hours-units mins-tens mins-units
colon (the two dots) = relay 8 of Module 1
SEGMENT MAP (matches your "clock frame numbered.jpg"):
relay 1 -> segment f (top-left) x.1
relay 2 -> segment a (top) x.2
relay 3 -> segment b (top-right) x.3
relay 4 -> segment g (middle) x.4
relay 5 -> segment e (bottom-left) x.5
relay 6 -> segment c (bottom-right) x.6
relay 7 -> segment d (bottom) x.7
relay 8 -> (Module 1 only) the colon dots 1.8
(relay 8 of modules 2,3,4 is unused)
BEHAVIOUR
- Normal: shows HH:MM, colon ON steady (NEVER blinks -> saves the
colon relay from millions of pointless switch cycles).
- At second 58: shows "EEb3", colon OFF (during active periods only).
- At second 59: shows the current period number, colon OFF (periods only).
During breaks/gaps/before P1/after P9 -> plain time.
- Outside 08:00-17:00: all 32 relays OFF - completely dark.
RELIABILITY ("long life") choices made on purpose:
- A relay is only switched when that segment actually changes state.
- The colon does not blink.
- All relays rest completely outside school hours (08:00-17:00).
- Hardware watchdog reboots the Mega automatically if it ever hangs.
- RTC is read defensively; garbage reads are ignored.
============================================================================ */
#include <Wire.h>
#include <RTClib.h>
#include <avr/wdt.h>
RTC_DS3231 rtc;
/* ----------------------------------------------------------------------------
1) RTC TIME SYNC - bulletproof two-stage system
STAGE 1 (primary, millisecond-accurate):
After every upload, run python3 set_rtc.py in Terminal.
The script sends the exact current UTC to the Arduino over Serial.
The Arduino sets the RTC from that - accurate to within 1 second.
Your Mac clock is NTP-synced so this is always correct.
STAGE 2 (automatic fallback):
If the Python script is NOT run within SERIAL_WAIT_MS milliseconds,
the Arduino falls back to the compile-time stamp from your Mac clock,
automatically converted from Brussels local time to UTC.
This is typically 10-20 seconds behind real time - fine for a clock.
SERIAL_WAIT_MS How long to wait for the Python script (milliseconds).
5000 = 5 seconds.
---------------------------------------------------------------------------- */
#define SERIAL_WAIT_MS 5000UL
/* ----------------------------------------------------------------------------
2) RELAY POLARITY
The common blue 8-relay boards are ACTIVE LOW (IN pin LOW = relay ON).
If your board turns the bulb ON when the IN pin is HIGH, set this to false.
---------------------------------------------------------------------------- */
#define RELAY_ACTIVE_LOW true
/* ----------------------------------------------------------------------------
3) DISPLAY OPTIONS
---------------------------------------------------------------------------- */
#define BLANK_LEADING_HOUR_ZERO true // " 9:05" instead of "09:05"
```

```
const uint8_t EEB3_SECOND = 58; // second :58 shows "EEb3" (periods only)
const uint8_t PERIOD_SECOND = 59; // second :59 shows period label (periods only)
/* ------------------------------------------------------------------------
3b) OPERATING HOURS
Outside these hours ALL relays are switched OFF completely.
This gives the relay contacts a long rest every day and maximises
the lifespan of all 32 relays and the light bulbs.
Times in LOCAL Brussels time, minutes from midnight.
08:00 = 8*60 = 480 17:00 = 17*60 = 1020
------------------------------------------------------------------------ */
const int CLOCK_ON_MINS = 8 * 60; // 08:00 - relays wake up
const int CLOCK_OFF_MINS = 17 * 60; // 17:00 - relays go to sleep
/* ------------------------------------------------------------------------
4) PIN MAP - each module's 8 IN pins kept together in one neat block.
Module N: { IN1, IN2, IN3, IN4, IN5, IN6, IN7, IN8 }
(index 0 = IN1 = relay 1 = segment f ... see SEGMENT MAP above)
------------------------------------------------------------------------ */
const uint8_t PINS[4][8] = {
{ 22, 23, 24, 25, 26, 27, 28, 29 }, // Module 1 (hours tens) + colon on IN8
{ 30, 31, 32, 33, 34, 35, 36, 37 }, // Module 2 (hours units)
{ 38, 39, 40, 41, 42, 43, 44, 45 }, // Module 3 (minutes tens)
{ 46, 47, 48, 49, 50, 51, 52, 53 } // Module 4 (minutes units)
};
const uint8_t COLON_RELAY_INDEX = 7; // IN8 of module 1 (0-based index 7)
// a b c d e f g
const uint8_t SEG2RELAY[7] = {1, 2, 5, 6, 4, 0, 3};
/* ------------------------------------------------------------------------
5) BELL SCHEDULE (LOCAL Brussels time, minutes from midnight)
Edit freely. disp must be exactly 4 characters.
Break (10:55-11:15) is a gap - no entry - clock shows plain time.
------------------------------------------------------------------------ */
struct Period { int startMins; int endMins; const char* disp; };
Period schedule[] = {
{ 8 * 60 + 30, 9 * 60 + 15, "P 1" },
{ 9 * 60 + 20, 10 * 60 + 5, "P 2" },
{10 * 60 + 10, 10 * 60 + 55, "P 3" },
// 10:55 - 11:15 BREAK - gap in schedule, shows time only
{11 * 60 + 15, 12 * 60 + 0, "P 4" },
{12 * 60 + 5, 12 * 60 + 50, "P 5" },
{13 * 60 + 0, 13 * 60 + 45, "P 6" },
{13 * 60 + 50, 14 * 60 + 35, "P 7" },
{14 * 60 + 40, 15 * 60 + 25, "P 8" },
{15 * 60 + 30, 16 * 60 + 15, "P 9" }
// 16:15 - 17:00 after last period - shows time only
};
const int NUM_PERIODS = sizeof(schedule) / sizeof(schedule[0]);
const char* getActivePeriod(int localMins) {
for (int i = 0; i < NUM_PERIODS; i++)
if (localMins >= schedule[i].startMins && localMins < schedule[i].endMins)
return schedule[i].disp;
return nullptr;
}
/* ========================================================================
7-SEGMENT FONT
bit0=a bit1=b bit2=c bit3=d bit4=e bit5=f bit6=g
======================================================================== */
byte segMask(char ch) {
switch (ch) {
case '0': return 0b0111111;
case '1': return 0b0000110;
case '2': return 0b1011011;
case '3': return 0b1001111;
case '4': return 0b1100110;
case '5': return 0b1101101;
case '6': return 0b1111101;
case '7': return 0b0000111;
case '8': return 0b1111111;
case '9': return 0b1101111;
case 'E': return 0b1111001;
case 'b': return 0b1111100;
```

```
case 'P': return 0b1110011;
case 'r': return 0b1010000;
case 'c': return 0b1011000;
case 'd': return 0b1011110;
case '-': return 0b1000000;
case ' ':
default: return 0b0000000;
}
}
/* ============================================================================
EU / BRUSSELS DAYLIGHT-SAVING (rule based, no lookup table needed)
Summer time (CEST, UTC+2): last Sunday of March 01:00 UTC ->
last Sunday of October 01:00 UTC.
Otherwise winter time (CET, UTC+1).
============================================================================ */
uint8_t lastSundayDay(uint16_t year, uint8_t month) {
DateTime lastDay(year, month, 31, 0, 0, 0);
uint8_t dow = lastDay.dayOfTheWeek(); // 0 = Sunday
return 31 - dow;
}
bool isEuSummerTime(const DateTime& u) {
uint8_t m = u.month();
if (m < 3 || m > 10) return false;
if (m > 3 && m < 10) return true;
uint8_t ls = lastSundayDay(u.year(), m);
if (m == 3) {
if (u.day() > ls) return true;
if (u.day() < ls) return false;
return u.hour() >= 1;
} else {
if (u.day() < ls) return true;
if (u.day() > ls) return false;
return u.hour() < 1;
}
}
DateTime utcToBrussels(const DateTime& utc) {
int offsetHours = isEuSummerTime(utc) ? 2 : 1;
return utc + TimeSpan(0, offsetHours, 0, 0);
}
bool isLocalBrusselsSummerTime(const DateTime& local) {
uint8_t m = local.month();
if (m < 3 || m > 10) return false;
if (m > 3 && m < 10) return true;
uint8_t ls = lastSundayDay(local.year(), m);
if (m == 3) {
if (local.day() > ls) return true;
if (local.day() < ls) return false;
return local.hour() >= 2;
} else {
if (local.day() < ls) return true;
if (local.day() > ls) return false;
return local.hour() < 3;
}
}
DateTime brusselsLocalToUtc(const DateTime& local) {
int offsetHours = isLocalBrusselsSummerTime(local) ? 2 : 1;
return local - TimeSpan(0, offsetHours, 0, 0);
}
/* ============================================================================
RELAY OUTPUT with change-tracking (only switch when state actually changes)
============================================================================ */
bool relayState[4][8];
inline void writeRelay(uint8_t module, uint8_t idx, bool on) {
if (relayState[module][idx] == on) return;
relayState[module][idx] = on;
uint8_t level = (on == RELAY_ACTIVE_LOW) ? LOW : HIGH;
digitalWrite(PINS[module][idx], level);
}
// Turn every relay OFF - used outside operating hours.
// writeRelay() skips pins already off, so zero clicks once already dark.
```

```
void allRelaysOff() {
for (uint8_t m = 0; m < 4; m++)
for (uint8_t i = 0; i < 8; i++)
writeRelay(m, i, false);
}
void applyDigit(uint8_t module, char ch) {
byte mask = segMask(ch);
for (uint8_t seg = 0; seg < 7; seg++) {
bool on = mask & (1 << seg);
writeRelay(module, SEG2RELAY[seg], on);
}
}
void setColon(bool on) {
writeRelay(0, COLON_RELAY_INDEX, on);
}
void renderFrame(const char d[4], bool colonOn) {
for (uint8_t m = 0; m < 4; m++) applyDigit(m, d[m]);
setColon(colonOn);
}
/* ============================================================================
SETUP
============================================================================ */
void setup() {
Serial.begin(9600);
// Force all relay outputs OFF before enabling them (no boot glitch)
for (uint8_t m = 0; m < 4; m++) {
for (uint8_t i = 0; i < 8; i++) {
uint8_t offLevel = RELAY_ACTIVE_LOW ? HIGH : LOW;
digitalWrite(PINS[m][i], offLevel);
pinMode(PINS[m][i], OUTPUT);
digitalWrite(PINS[m][i], offLevel);
relayState[m][i] = false;
}
}
Wire.begin();
if (!rtc.begin()) {
Serial.println(F("RTC not found! Check wiring (SDA=20, SCL=21)."));
}
Serial.println(F("READY"));
bool synced = false;
unsigned long waitStart = millis();
while ((millis() - waitStart) < SERIAL_WAIT_MS) {
if (Serial.available()) {
char cmd = Serial.read();
if (cmd == 'T') {
unsigned long unixUtc = Serial.parseInt();
if (unixUtc > 1700000000UL) {
rtc.adjust(DateTime(unixUtc));
Serial.print(F("RTC SET via script -> UTC: "));
Serial.println(DateTime(unixUtc).timestamp());
synced = true;
break;
}
}
}
}
if (!synced) {
DateTime compileLocal(F(__DATE__), F(__TIME__));
DateTime utcNow = brusselsLocalToUtc(compileLocal);
rtc.adjust(utcNow);
Serial.print(F("RTC FALLBACK (compile time) -> UTC: "));
Serial.println(utcNow.timestamp());
Serial.println(F("Run python3 set_rtc.py for exact time next upload."));
}
wdt_enable(WDTO_8S);
}
/* ============================================================================
LOOP
============================================================================ */
int lastRenderedSecondKey = -1;
```

```
void loop() {
wdt_reset();
static unsigned long lastTick = 0;
if (millis() - lastTick < 200) return;
lastTick = millis();
DateTime utc = rtc.now();
if (utc.year() < 2024 || utc.year() > 2099) return;
DateTime local = utcToBrussels(utc);
uint8_t hh = local.hour();
uint8_t mm = local.minute();
uint8_t ss = local.second();
int localMins = hh * 60 + mm;
// Outside 08:00-17:00: all relays off. Sentinel -2 prevents repeat clicks.
if (localMins < CLOCK_ON_MINS || localMins >= CLOCK_OFF_MINS) {
if (lastRenderedSecondKey != -2) {
lastRenderedSecondKey = -2;
allRelaysOff();
Serial.println(F("Clock OFF (outside operating hours)."));
}
return;
}
// EEb3/:59 only during active periods. Breaks/gaps/before P1/after P9
// show plain time the entire minute.
char frame[4];
bool colonOn;
const char* activePeriod = getActivePeriod(localMins);
bool periodActive = (activePeriod != nullptr);
if (periodActive && ss == EEB3_SECOND) {
frame[0] = 'E'; frame[1] = 'E'; frame[2] = 'b'; frame[3] = '3';
colonOn = false;
}
else if (periodActive && ss == PERIOD_SECOND) {
frame[0] = activePeriod[0]; frame[1] = activePeriod[1];
frame[2] = activePeriod[2]; frame[3] = activePeriod[3];
colonOn = false;
}
else {
frame[0] = (BLANK_LEADING_HOUR_ZERO && hh < 10) ? ' ' : ('0' + hh / 10);
frame[1] = '0' + hh % 10;
frame[2] = '0' + mm / 10;
frame[3] = '0' + mm % 10;
colonOn = true;
}
int key = (frame[0] << 24) ^ (frame[1] << 16) ^ (frame[2] << 8)
^ frame[3] ^ (colonOn ? 0x100000 : 0);
if (key != lastRenderedSecondKey) {
lastRenderedSecondKey = key;
renderFrame(frame, colonOn);
Serial.print(F("UTC ")); Serial.print(utc.timestamp());
Serial.print(F(" Local "));
Serial.print(hh); Serial.print(':');
if (mm < 10) Serial.print('0'); Serial.print(mm);
Serial.print(F(" period=")); Serial.print(periodActive ? activePeriod : "none");
Serial.print(F(" showing "));
Serial.write((const uint8_t*)frame, 4);
Serial.print(F(" colon=")); Serial.println(colonOn ? "ON" : "off");
}
}
```

11. QUICK REFERENCE

All pin assignments

	Mega pins
	20, 21
	22 23 24 25 26 27 28 29
	30 31 32 33 34 35 36 37
	38 39 40 41 42 43 44 45
	46 47 48 49 50 51 52 53

Configurable constants in code

	Default	Change when
	true	Relay boards fire on HIGH instead of LOW (reverse polarity board
	true	You prefer 09:05 displayed instead of 9:05
	58	Moving the EEb3 flash to a different second
	59	Moving the period label to a different second
	8*60 = 480	Time when relays wake up (08:00 by default)
	17*60 = 1020	Time when all relays go to sleep (17:00 by default)
	5000	Milliseconds to wait for the Python time-setter script at boot

---

Prepared by Taqi Abbas | European School of Brussels, Ixelles